GUIDE 51 · OPERATE

See QueryAPIGate in your traces

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=51-see-queryapigate-in-your-traces. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** one trace from your app, through QueryAPIGate, to the database - in Jaeger here, or any backend that takes OpenTelemetry (Tempo, Honeycomb, Datadog, Grafana Cloud) - showing where the time went.

> **Experimental** - tracing is new in 0.23 and its span names and attributes may change in any minor release, always noted in the changelog (what that means).

Every command and result below was run against a real server (gunicorn, one worker, 8 threads), a PostgreSQL 16 `orders` table of 3,000 rows behind the saved query `orders_by_customer`, and Jaeger 2.5 as the collector.

Step 1: A collector

Jaeger takes OTLP itself, on port 4318:

```shell
docker run -d --name jaeger -p 16686:16686 -p 4318:4318 jaegertracing/jaeger:2.5.0
```

(In production this is usually an OpenTelemetry Collector, forwarding to your backend.)

Step 2: Turn tracing on

```shell
pip install "queryapigate[otel]"
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318 OTEL_SERVICE_NAME=queryapigate \
  gunicorn --bind 127.0.0.1:5000 --workers 1 --worker-class gthread --threads 8
"queryapigate.app:create_app()"
```

The log says it's on - *Experimental: Tracing with OpenTelemetry (OTEL_EXPORTER_OTLP_ENDPOINT) - may change in any minor release* - and **Settings → Tracing** shows the endpoint, the service name and the sampler. Every other OpenTelemetry variable works as it does for any service (`OTEL_RESOURCE_ATTRIBUTES`, `OTEL_EXPORTER_OTLP_HEADERS` for a backend's token, …). Without the extra installed, setting the endpoint stops the server: *An OTLP endpoint is set … run* `pip install "queryapigate[otel]"`.

Step 3: Call it from inside your app's trace

Your app sends its trace context in the `traceparent` header - OpenTelemetry's `propagate.inject()` does it, as do most HTTP clients' instrumentation:

```python
with tracer.start_as_current_span('GET /customers/{id}/orders'):
    headers = {'X-API-Key': key}
    propagate.inject(headers)  # adds traceparent
    urllib.request.urlopen(urllib.request.Request(
        'http://127.0.0.1:5000/q/orders_by_customer?customer=acme', headers=headers))
```

Step 4: One trace, end to end

In Jaeger (`http://localhost:16686`), the app's trace - here `5d1b7946cc208d89e08d0758967125e2` - has QueryAPIGate's spans inside it:

```text
shop-backend: GET /customers/{id}/orders   39.4 ms
  queryapigate: GET /q/<name>              12.8 ms   http.route=/q/<name>  status 200  caller=shop-
backend
    queryapigate: query orders_by_customer 11.7 ms   queryapigate.rows=10
      queryapigate: SELECT shop            10.1 ms   db.system.name=postgresql  db.namespace=postgres
                                                     server.address=localhost
db.response.returned_rows=10
      queryapigate: serialize json          0.1 ms   queryapigate.format=json  queryapigate.bytes=226
```

So of 12.8 ms in QueryAPIGate, 10.1 went to PostgreSQL. Each span records only what the list says. The parameter (`acme`), the statement, the API key and the database password appear nowhere in the stored trace (checked by searching it), and that holds for every span: tests fail if one ever does.

From the other side: the same `trace_id` is in the run's history entry (`GET /api/v1/history`), in an error's body beside `request_id`, and in each JSON log line (`QUERYAPIGATE_JSON_LOGS=1`) with its `span_id` - so a slow run, an error a caller reports, or a log line leads straight to its trace.

What it costs

2,000 sequential requests to that query (100 rows each, one keep-alive connection, after 200 to warm up), each setting run twice:

	MEDIAN	P95	P99
tracing off	3.42–3.80 ms	5.89–6.79 ms	6.95–9.41 ms
every request traced	4.50–4.79 ms	7.28–7.35 ms	10.85–11.63 ms
10% of requests traced	3.27–3.58 ms	5.37–5.40 ms	6.26–6.40 ms

Tracing every request cost about 1 ms each - five spans, and encoding and sending them in the background while requests run. Sampling a tenth cost nothing measurable:

```shell
OTEL_TRACES_SAMPLER=parentbased_traceidratio OTEL_TRACES_SAMPLER_ARG=0.1
```

`parentbased_...` follows the caller's decision when it sends one and samples a tenth of the rest. Plain `traceidratio` ignores the caller's - so a client can't ask for its every request to be traced.

Other front doors

- **MCP:** an agent's `tools/call` is a server span, `tools/call <tool>`, with the query and database spans inside it, continuing the client's `traceparent` when it sends one.
- **Exports:** a saved export run from the API is inside the request's trace. `queryapigate export` and `queryapigate exports run` start their own - or continue `TRACEPARENT` from the environment, so a scheduler that sets it (Airflow, a CI job) finds the run in its own trace.
- **Live events:** opening a stream is one span, `GET /events` - not one per event; a stream can stay open for hours.

Next steps

- Tracing reference - every span and attribute.
- Read the threat model - what goes to a collector, and who reads it.
- Trace a request in the logs - the request id, without a tracing backend.
- Wire up Prometheus and Grafana - metrics stay on `/metrics`.